Cloud-Native Healthcare Data Management System

Phase 2 — Implementation Report

Course: Cloud Computing (CSIZG527) · BITS Pilani, M.Tech Program **Group:** 23 **Members:** Karan Rawat · Vikas Kumar · Kriti Tripathi · Avanish Pratap Singh **Instructor:** Prof. Shwetha Vittal **Submission date:** 02 May 2026

1. Abstract

Mid-size hospital chains in India struggle with fragmented on-premise IT that cannot absorb peak OPD load, exposes patient data to compliance risk, and consumes more than half of IT staff time on maintenance. We design and implement a **cloud-native Healthcare Data Management System (HDMS)** on AWS that addresses all six Phase 1 objectives. The system is built as a single FastAPI codebase whose storage, database, logging and notification layers are pluggable through configuration, allowing identical code to run locally (SQLite + filesystem) and on AWS (RDS PostgreSQL + S3 + Lambda). The cloud topology comprises a VPC with public/private subnets, an Application Load Balancer fronting an EC2 Auto Scaling Group, a Multi-AZ RDS PostgreSQL instance, a KMS-encrypted S3 bucket for medical files, an EventBridge-triggered Lambda that publishes appointment reminders to SNS, CloudTrail for full audit logging, and CloudWatch for metrics and alarms. The entire stack is captured as Terraform Infrastructure-as-Code (≈ 600 lines) and validated by a 29-test pytest suite that exercises both the business logic and the cloud abstractions using the `moto` mock-AWS library, so no live AWS credentials are required for CI.

2. Survey

Approach	Strengths	Weaknesses	Source
On-premise legacy	Full hardware control, predictable latency	High CapEx, can't auto-scale, manual patching	Gartner Healthcare Cloud Report 2024
Hybrid cloud (current norm, ~60% of Indian hospitals)	Gradual modernisation, partial elasticity	Multi-vendor complexity, inconsistent security	NASSCOM Digital Health 2024
Cloud-native (this project)	Auto-scaling, unified data, pay-per-use, built-in compliance primitives	Cloud lock-in, requires cloud-skilled team	AWS Healthcare Whitepapers; NITI Aayog Digital Health Blueprint

Open-source academic precedents such as **OpenMRS** and **OpenEMR** demonstrate the viability of FastAPI/PostgreSQL stacks for EHR workloads but stop at single-tenant deployments. Commercial offerings (AWS for Healthcare, Azure Health Data Services, GCP Cloud Healthcare API) provide managed FHIR services but at price points unsuitable for mid-size Indian chains. Our design borrows the open-source data model and bolts on the managed-AWS scalability layer.

3. Problem Statement

> A mid-size hospital chain in India operates on fragmented, on-premise IT > with partial cloud adoption. Peak OPD hours overload servers, branch-level > databases prevent unified patient care, encryption and access logging gaps > fail DISHA / ABDM audits, and over-provisioned hardware idles 70 % of the > time. There is no cloud-native architecture providing auto-scaling, > unified patient data management, real-time analytics, and > compliance-ready security — resulting in degraded patient care and > operational inefficiency.

The Phase 1 statement is retained verbatim; Phase 2 supplies the implementation that operationalises it.

4. Objectives (refined)

ID	Objective (Phase 1)	Phase 2 Implementation
O1	Cloud-native architecture using AWS that auto-scales to 3× peak traffic	EC2 Auto Scaling Group (min 1, max 3) behind ALB, TargetTracking on 60 % avg CPU
O2	Unified patient data layer eliminating silos, < 200 ms query latency	Single RDS PostgreSQL Multi-AZ in private subnets, indexed PKs, connection pooling via SQLAlchemy
O3	Security & compliance — IAM RBAC, KMS, CloudTrail, HIPAA/DISHA-aligned	IAM instance + Lambda roles (least-privilege), KMS CMK encrypting RDS + S3, S3 SSE-KMS + versioning + 7-yr lifecycle, CloudTrail multi-region with log file validation, IMDSv2-only EC2
O4	Serverless analytics & event processing	Lambda (Python 3.11) on EventBridge cron (15 min) reads upcoming appointments → SNS topic → email/SMS
O5	40–60 % infra cost reduction via pay-per-use	t3.micro instances, single NAT, S3 lifecycle to IA→Glacier, Lambda billed per-invocation, RDS scales-down friendly
O6	99.9 % availability via Multi-AZ + ELB	RDS Multi-AZ failover, ASG across 2 AZs, ALB health checks (`GET /`), CloudWatch alarms wired to SNS

5. Outcomes

5.1 Software Deliverables

Deliverable	Path	Lines
FastAPI application (Phase 1 + Phase 2)	`app/`	~1 800
Pluggable storage layer	`app/storage.py`	116
CloudWatch JSON logging middleware	`app/logging_middleware.py`	90
Appointment reminder Lambda	`app/lambda_handlers/appointment_reminder.py`	115
Terraform IaC (10 .tf files + user-data)	`infra/terraform/`	~ 600
Dockerfile	`Dockerfile`	25
Test suite	`tests/`	~ 600

5.2 Validation

```
$ python -m pytest tests/ -v
============================== 29 passed in 28.30s ==============================
```

Suite	Tests	What it proves
`test_auth.py`	6	JWT issue/verify, RBAC denials, password hashing
`test_patients.py`	7	Patient CRUD + soft-delete + pagination
`test_appointments.py`	4	Booking, status transitions, role-gated mutations
`test_storage.py`	4	LocalStorage round-trip, S3Storage round-trip via `moto`, factory selection
`test_lambda_reminder.py`	3	Time-window filter, dry-run when SNS not configured, SNS publish path
`test_logging_middleware.py`	3	JSON formatter shape, request ID propagation, idempotent configuration
`test_upload_e2e.py`	2	End-to-end PDF upload via the storage abstraction; rejection of bad ext.

Cloud abstractions are validated **without live AWS credentials** by mocking S3/SNS through `moto`, which is the AWS-recommended approach for CI.

5.3 Cloud Topology (deployable)

The Terraform stack provisions, in a single `terraform apply`:

• 1 VPC, 2 public + 2 private subnets across 2 AZs, 1 IGW, 1 NAT GW

• 1 KMS Customer-Managed Key with rotation enabled (encrypts both RDS storage and S3 objects)

• 1 RDS PostgreSQL 15 Multi-AZ (`db.t3.micro`, encrypted, Performance Insights on)

• 1 S3 bucket: SSE-KMS, versioned, public access blocked, 30 d → IA, 90 d → Glacier, 7 y expiry

• 1 ALB + target group + HTTP listener

• 1 Launch template + Auto Scaling Group (`t3.micro`, IMDSv2-only, EC2 instance role)

• 1 Lambda function packaged from the same `app/` tree, on EventBridge `rate(15 minutes)`

• 1 SNS topic (KMS-encrypted) for reminders **and** alarm fan-out

• 1 CloudTrail (multi-region, log-file-validation, KMS-encrypted)

• 1 CloudWatch log group `/aws/ec2/healthdms` (KMS-encrypted, 30 d retention)

• 2 CloudWatch alarms (ALB 5xx > 5/min, ASG CPU > 80 % for 3 min)

5.4 Service Mapping (Project policy: AWS-first; no other cloud used)

The Phase 1 service-mapping table is preserved at `docs/04-aws-service-mapping.md`. All services referenced in Phase 2 are AWS native.

6. References

1. AWS Documentation — EC2 Auto Scaling, RDS, S3, Lambda, IAM, KMS, CloudTrail, CloudWatch (2024–2026 editions). 2. HashiCorp Terraform — AWS Provider docs, `~> 5.50`. 3. NIST SP 800-66 Rev. 2 — Implementing the HIPAA Security Rule (2024). 4. Government of India — *Digital Information Security in Healthcare Act (DISHA) draft, 2018* and *Ayushman Bharat Digital Mission* implementation guides. 5.

NASSCOM — *Digital Health: Indian Industry Outlook 2024*. 6. Gartner — *Healthcare Cloud Report 2024*. 7. NITI Aayog — *National Digital Health Blueprint*, 2019. 8. FastAPI documentation — https://fastapi.tiangolo.com/ 9. SQLAlchemy 2.0 documentation — https://docs.sqlalchemy.org/en/20/ 10. `moto` library documentation — https://docs.getmoto.org/

7. Short Reflection (mandatory rubric section)

7.1 Workplace relevance

The four group members work in software, banking and analytics roles where cloud migrations are a daily concern. The exact pattern we built — pluggable storage so the same app runs on a developer laptop and on AWS — mirrors how our employers ship services through dev → staging → prod. The Multi-AZ + ALB + ASG topology is the same pattern we have observed in production fintech and SaaS deployments; applying it to a healthcare context made the constraints (PHI sensitivity, HIPAA/DISHA, retention) explicit instead of implicit.

7.2 Cloud concepts learned

• **Service composition over service selection.** Each AWS primitive is

trivial in isolation; the value comes from composing them with IAM, KMS, security groups and VPC routing so that data never traverses the public internet.

• **IaC as the unit of review.** Writing the entire stack as Terraform

forced us to confront the blast radius of every resource — particularly KMS keys, IAM policies and the RDS Multi-AZ failover semantics — in a way that AWS-Console click-ops would have hidden.

• **Mocked-AWS testing (`moto`).** Realistic unit tests for cloud calls

without burning real credentials let us iterate quickly and keep CI free.

• **Cost is a design input, not an afterthought.** Choosing one NAT

Gateway, `t3.micro` instances and S3 lifecycle transitions cut the monthly cost from an estimated ~ ■ 25 000 to under ■ 8 000 for the demo footprint, without compromising the architectural objectives.

7.3 Trade-offs made and why

Decision	Alternative	Why we chose it
RDS PostgreSQL	DynamoDB	Patient/appointment data is highly relational; joins matter.
Single NAT Gateway	NAT per AZ	Cost — academic project; AZ-failure tolerance acceptable for demo.
`t3.micro` everywhere	Larger burstable / Graviton	Stays inside AWS Free Tier limits where possible.
Lambda on EventBridge cron	EC2 cronjob inside the app	Decouples reminder schedule from app uptime; pay-per-invocation.
Terraform	AWS CDK / CloudFormation	Team familiarity; multi-cloud-friendly even though we stayed on AWS.
Pluggable `StorageBackend` Protocol	Hard-code `boto3` in the router	Phase 1 still runs unchanged; tests stay fast; SOLID `D` (DIP).
Skip HTTPS / ACM in IaC	Provision ACM cert + alias record	Domain not assumed; `terraform apply` stays self-contained.

Decision	Alternative	Why we chose it
Defer WAF	Add WAF + managed rules	Out of marks-bearing scope; documented in Production Hardening notes.

7.4 What we would do differently next

• Move RDS credentials into AWS Secrets Manager and have the EC2 user-data

fetch them at boot, so the master password never appears in Terraform state.

• Add an **integration smoke test** that runs against `terraform outputs`

after `apply` (curl ALB, upload to S3, check the object lands).

• Replace the EC2 ASG with **App Runner** or **ECS Fargate** for an even

smaller operational surface — the Dockerfile we ship would slot in unchanged.

End of report.